

DL-RSIM: A Simulation Framework to Enable Reliable ReRAM-based Accelerators for Deep Learning

Meng-Yao Lin*, Hsiang-Yun Cheng[†], Wei-Ting Lin*, Tzu-Hsien Yang*, I-Ching Tseng*, Chia-Lin Yang*[†],
Han-Wen Hu[§], Hung-Sheng Chang[§], Hsiang-Pang Li[§], Meng-Fan Chang^{‡*}

*National Taiwan University, [†]Academia Sinica,

[§]Macronix International Co., Ltd., [‡]National Tsing Hua University

r05922028@ntu.edu.tw, hycheng@citi.sinica.edu.tw, {r06944031, r06922094, b04902016}@ntu.edu.tw, yangc@csie.ntu.edu.tw,
{paddingtonhu, billchang, sbli}@mxic.com.tw, mfchang@ee.nthu.edu.tw

ABSTRACT

Memristor-based deep learning accelerators provide a promising solution to improve the energy efficiency of neuromorphic computing systems. However, the electrical properties and crossbar structure of memristors make these accelerators error-prone. To enable reliable memristor-based accelerators, a simulation platform is needed to precisely analyze the impact of non-ideal circuit and device properties on the inference accuracy. In this paper, we propose a flexible simulation framework, DL-RSIM, to tackle this challenge. DL-RSIM simulates the error rates of every sum-of-products computation in the memristor-based accelerator and injects the errors in the targeted TensorFlow-based neural network model. A rich set of reliability impact factors are explored by DL-RSIM, and it can be incorporated with any deep learning neural network implemented by TensorFlow. Using three representative convolutional neural networks as case studies, we show that DL-RSIM can guide chip designers to choose a reliability-friendly design option and develop reliability optimization techniques.

1 INTRODUCTION

Deep learning and neuromorphic algorithms have attracted a lot of interest in recent years as they can provide optimal solutions for many classification and regression tasks, including image/speech recognition, data mining, and natural language processing. These neuromorphic algorithms often cost much more memory and computing resources than traditional methods, incurring tremendous data movements [4]. In conventional von Neumann architecture where computation and data storage are separated, such data movements become critical performance and energy bottleneck [8].

*This work is supported in part by research grants from the Ministry of Science and Technology of Taiwan (MOST-105-2221-E-002-156-MY2, MOST107-2218-E-001-004-MY3), and sponsored by Macronix Inc., Hsin-chu, Taiwan (97-SC39).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '18, November 5–8, 2018, San Diego, CA, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-5950-4/18/11...\$15.00

<https://doi.org/10.1145/3240765.3240800>

The innovation of memristor-based computing-in-memory accelerator provides a promising solution to reduce data movements and boost the energy efficiency of neuromorphic computation. Recent work demonstrated that metal-oxide resistive random access memory (ReRAM) [11] can be used to build a crossbar structure to efficiently perform matrix-vector multiplications in convolution and fully-connected layers of deep neural networks [7]. With the capability of performing arithmetic operations beyond data storage, ReRAM-based computing-in-memory accelerators can provide significant energy savings comparing to CPU or GPU solutions [3, 9].

While the electrical properties and crossbar structure of ReRAM enable energy-efficient neuromorphic computing, these properties also make the ReRAM-based accelerators error-prone. When performing matrix-vector multiplication in a ReRAM crossbar array, multiple wordlines are activated concurrently and the currents flowed through different cells in the same bitline are accumulated to get the sum-of-products result. Even if none of cells in a bitline is in error, the sum-of-products result read out by the sense amplifier may be incorrect [5] mainly due to (1) the accumulation of read noise from each activated cell in the bitline and (2) the imperfect sense amplifier with limited resolution. Since many factors can affect the reliability of ReRAM-based accelerators, a simulation platform is needed to estimate the impact of current sensing errors and hardware constraints on the inference accuracy of the targeted neural networks.

Prior work has studied the performance and energy efficiency of ReRAM-based deep learning accelerators [3, 9] and proposed simulation tools to facilitate design space exploration [1, 12], but none of the existing simulation platforms completely supports flexible and precise error rate analysis. The non-ideal circuit and device properties that cause the current sensing errors are usually ignored in prior system-level work [3, 9]. Some circuit-level [1] and behavior-level [12] simulation platforms take the impact of hardware errors into consideration. However, the circuit-level tool [1] is hard to be integrated with various neural networks while the behavior-level tool [12] provides only imprecise analysis on inference accuracy. As a result, a flexible and accurate simulation platform that fully explore the impact of hardware errors on various neural network models is highly demanded.

In this paper, we propose a flexible simulation framework, DL-RSIM, for ReRAM-based deep learning accelerators to analyze the impact of non-ideal circuit and device properties on the inference accuracy of various neuromorphic computing

workloads. DL-RSIM simulates the error rates of every sum-of-products computation in the memristor-based deep learning accelerator, and injects the errors in the TensorFlow-based neural network models. The main contributions of this work can be summarized as follows:

- DL-RSIM can be incorporated with any deep learning neural network implemented by TensorFlow, and a rich set of impact factors, i.e., non-ideal circuit and device properties in ReRAM-based accelerators, are explored when estimating the inference accuracy.
- DL-RSIM can be utilized to explore the design space of ReRAM-based deep learning accelerators for reliability optimization. The simulation results can guide chip designers to choose a better design option at an early design stage.
- We demonstrate how to use DL-RSIM to develop reliability optimization techniques by proposing and evaluating a workload-dependent sensing scheme. Computer architects can use DL-RSIM to analyze which part of the neural network is more vulnerable to errors and design optimization techniques accordingly.

2 BACKGROUND

2.1 Convolutional Neural Network

Convolutional neural network (CNN) is the key component of many deep learning applications. Figure 1 shows three representative types of layers in CNNs: convolution, pooling, and fully-connected layer. A typical CNN starts with multiple convolution layers to extract feature maps, and pooling layers are interleaved between different convolution layers to reduce the size of feature maps through performing subsampling. At the end of the CNN, one or multiple fully-connected layers are used to extract feature maps and correlate the feature maps to the target objects. In the convolution layer, a set of matrix-vector multiplications are conducted to convolute filters with input feature maps from the previous layer to generate output feature maps for the next layer. Suppose that there are C input channels and K $X \times Y$ filters, the (a,b) data at the k^{th} output feature map is calculated by the following equation with many sum-of-products computation:

$$Output(k, a, b) = \beta \left(\sum_{c=0}^{C-1} \sum_{y=0}^{Y-1} \sum_{x=0}^{X-1} Input(c, a+x, b+y) * Filter(k, x, y) \right) \quad (1)$$

where β is a non-linear activation function, such as sigmoid or ReLU. The fully-connected layer can be viewed as a special case of a convolution with the largest possible filter size (i.e., the size of the feature map). ReRAM-based deep learning accelerators primarily target on the acceleration of convolution and fully-connected layers, as these layers involve a significant number of matrix-vector multiplications.

2.2 ReRAM Crossbar Array

ReRAM is a type of non-volatile memory that stores data by changing cell resistances. The crossbar structure of ReRAM allows it to naturally compute matrix-vector multiplication in one constant time step. As illustrated in Figure 2(a), each ReRAM cell is located at the cross point of a wordline and a bitline. If input voltage V_i is applied to the i -th wordline, the current passed through the ReRAM cell with resistor

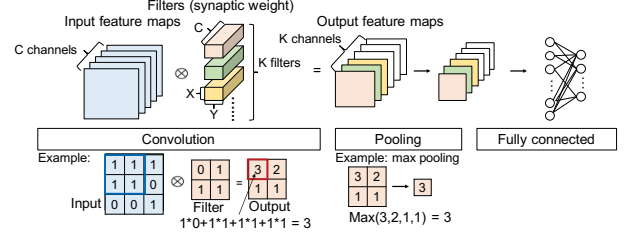


Figure 1: Convolutional neural network (CNN).

R_i is equal to V_i/R_i , and the total current flows through a bitline (I_{BL}) is $\sum V_i/R_i$. Thus, by storing filter weights as the conductance of ReRAM cells ($1/R_i$) and converting input feature maps into input voltage signals, we can get the dot-product results (output feature maps) at the end of the bitlines, as shown in Figure 2(b). With the computing-in-memory capability, ReRAM crossbar array provides a promising solution to accelerate CNNs.

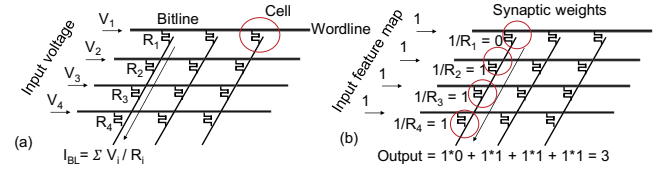


Figure 2: ReRAM crossbar for matrix-vector multiplication: (a) crossbar structure (b) neural network mapping.

3 RERAM-BASED DEEP LEARNING ACCELERATOR ARCHITECTURE

We first present the architecture of typical ReRAM-based deep learning accelerators. As shown in Figure 3, the ReRAM-based deep learning accelerator is composed of multiple computation units (CUs) connected with on-chip interconnects. Each CU has multiple crossbar arrays, which are responsible for the acceleration of matrix-vector multiplications in convolution and fully-connected layers. A wordline driver (WLD), such as a digital-to-analog converter (DAC) or an inverter [9], is connected to each wordline of the ReRAM crossbar array, in order to convert the input feature map data to input voltage. The accumulated currents on bitlines (sum-of-products results) are read out by multi-level sense amplifiers (SAs). As a multi-level SA consumes a significant amount of power and chip area [1, 9], each SA is shared by multiple bitlines. In addition to the crossbar arrays, there is one on-chip buffer in each CU for temporarily storing input and output feature maps. A non-linear function (ReLU) unit and a pooling unit are also included in the CU to support the implementation of non-linear function and pooling layer in the neural network.

When performing a matrix-vector multiplication (dot-product computation), the input vector is encoded using multiple clock cycles, as an area- and energy-efficient WLD usually has limited resolution [9]. Since a practical ReRAM cell can store only a limited number of bits [7, 9], each data in the original filter weight is also partitioned and mapped onto different bitlines. Ideally, all the wordlines in the crossbar array can be activated concurrently in one single cycle.

However, only a limited number of wordlines can be activated concurrently in a practical design [2, 10], as activating a massive number of wordlines consumes too much power and it is hard to accurately read out the current accumulated from too many cells at the SA. Thus, each original dot-product computation is split into the dot-product computation of multiple operation units (OUs), as shown in Figure 3, with at most S_{OU} wordlines activated at one single cycle. The dot-product result of each OU are added together and temporarily stored at the register connected to the SA. After summing up the dot-product result of different OUs that share the same set of bitlines, a shift-and-add circuit is used to assemble the final result based on the bit position of input and synaptic weight.

Among these components in the CU, the inference accuracy of the neural network is mainly affected by the non-ideal properties and design constraints of ReRAM cell, operation unit, and sense amplifier. In the following subsections, we discuss the impact of these three components on sum-of-products error rates per bitline, and use an example to show how these errors and hardware constraints affect convolution computation in neural networks.

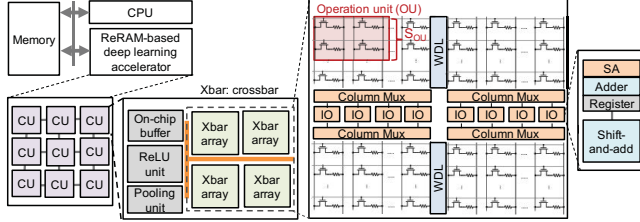


Figure 3: ReRAM-based accelerator architecture.

3.1 ReRAM Cell

The statistical randomness of ReRAM cell resistance is a critical issue when aiming to design a reliable ReRAM-based deep learning accelerator. Figure 4(a) shows the ReRAM cell structure, with a metal-oxide material sandwiched between two metal layers of electrodes. When a voltage is applied across a ReRAM cell, a conductive filament made out of oxygen vacancies is either formed (LRS, low resistance state) or ruptured (HRS, high resistance state) depending on the voltage polarity. Due to the randomness of the oxygen vacancy generation, the shape of the conductive filament varies from cycle to cycle even under the same programming condition. This resistance variability of ReRAM greatly impacts the error rate of matrix-vector multiplication. Recent work shows that the resistance histograms for both LRS and HRS follow the lognormal distribution [6]. As shown in Figure 4(b), the resistance distribution is mainly determined by the mean resistance of on-state (R_{on}), the resistance ratio between on and off state ($R\text{-ratio} = R_{off}/R_{on}$), and the resistance deviation (σ) of each state. When the resistance distributions of LRS and HRS are overlapped, it is hard to distinguish between these two states at the overlapped region. As a result, read error is more likely to occur on a cell with smaller R-ratio and/or larger σ due to the larger overlapped resistance region.

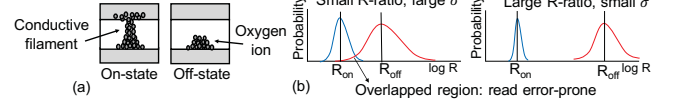


Figure 4: ReRAM (a) cell structure and (b) resistance distribution. σ : resistance deviation.

3.2 Operation Unit

In addition to the statistical randomness of ReRAM cell resistance, the size of the operation unit (S_{OU}) also affects the error rate of sum-of-products computation. When multiple wordlines are activated simultaneously, the output current (I_{BL}) of a bitline is equal to the summation of all activated cell currents. Since the resistance of ReRAM cells follows a lognormal distribution, the accumulated current on a bitline also follows a statistical distribution. Higher number of activated wordlines implies the accumulation of more cell resistance variance, leading to larger overlapped regions between neighbor states (sum-of-products values) in the accumulated current distribution and higher error rates. Thus, practical ReRAM crossbar designs [2, 10] restrict the maximum number of wordlines that can be activated concurrently (S_{OU}) to avoid serious accuracy degradation. Within an OU, the actual number of activated wordlines depends on the number of non-zeros inputs, as a wordline is turned off if the input is zero. If there are many non-zero values in the feature maps of the targeted neural network (i.e., many wordlines need to be activated), reducing S_{OU} degrades performance but can help to reduce bitline sensing error rates.

$$\begin{aligned} (I_R[n+1] - I_{BL}) - (I_{BL} - I_R[n]) &\geq dV_{SA}/\alpha \rightarrow \text{output } n \\ (I_R[n+1] - I_{BL}) - (I_{BL} - I_R[n]) &< -dV_{SA}/\alpha \rightarrow \text{output } n+1 \end{aligned}$$

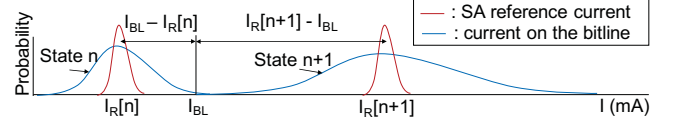


Figure 5: Dual reference sensing method.

3.3 Sense Amplifier

The design of sense amplifier affects the read sensing error rate of the sum-of-products result at each bitlines. Recent work showed that dual-reference sensing method can provide larger sensing margin and lower error rate than conventional sensing scheme [2]. As shown in Figure 5, when dual-reference sensing scheme is applied, the distance between the accumulated current on the bitline (I_{BL}) and the reference current of the higher state ($I_R[n+1]$) is compared with the distance between I_{BL} and the reference current of the lower state ($I_R[n]$) to determine the sum-of-products result. Considering the impact of SA safety guard band delta voltage (dV_{SA}), which represents the minimum voltage requirement to distinguish different states and is related to process variation, I_{BL} is sensed as the lower state if

$$\alpha * [(I_R[n+1] - I_{BL}) - (I_{BL} - I_R[n])] \geq dV_{SA} \quad (2)$$

where α is a constant that equals to the sensing speed of the SA divided by the SA input node capacitance. Otherwise, I_{BL} is sensed as the higher state if

$$\alpha * [(I_R[n+1] - I_{BL}) - (I_{BL} - I_R[n])] < -dV_{SA} \quad (3)$$

Larger dV_{SA} means it is more likely to sense a wrong value, as the correctness of SA is only guaranteed when the difference between $I_R[n+1] - I_{BL}$ and $I_{BL} - I_R[n]$ is larger than dV_{SA}/α . In addition, adjusting $I_R[n]$ and $I_R[n+1]$ based on workload characteristics can help to improve the inference accuracy of the targeted neural network. The resolution of SA also affects the error rate. A sense amplifier with lower resolution consumes less power and provides faster sensing speed [10], but the sum-of-products error rate increases as some bits (states) are truncated.

3.4 Dot-Product Computation under Hardware Constraints

To sum up the hardware constraints and non-ideal circuit/device properties mentioned above, we use an example to show how these limitations affect the process of dot-product computation in neural networks. Figure 6(a) represents the original dot-product computation. Due to the limited WLD resolution, bits-per-cell, and OU size, the original feature maps and filters are first partitioned into smaller OU-based bit-level computation units. We call this process **Decomposition**. As shown in Figure 6(b), the feature map is divided based on the WLD resolution (1 bit in this example). For example, the first data, 2, in the feature map is partitioned into 1 (MSB) and 0 (LSB) and 0 (LSB). The original filter is also partitioned into MSB and LSB columns based on the bits-per-cell (1 bit in this example). Since only a limited number of wordlines can be activated concurrently, the computation should be split into several OU-based portions. Suppose that $S_{OU}=2$, the MSB and LSB of the feature map and filter are further split into two portions.

After decomposition, **OU-based dot-product computation** is performed, as shown in Figure 6(c). For example, at the first cycle, the dot-product of the LSBs of the upper-half feature map (0,1) and the weights stored in OU1 is computed, generating the first output feature map (1,1). During the dot-product computation, the unstable cell resistance and non-ideal SA may generate error results, as shown in Figure 6(d). In this example, the fourth output feature map is wrongly sensed as (1,1) instead of original (1,0).

Finally, the OU-based bit-level results need to be resembled to get the final result of the dot-product computation and pass to the next neural network layer. We call this process **Composition**, as shown in Figure 6(e). During Composition, the dot-product outputs of the same bit-position in the input feature map but different OUs are first added together. For example, the first two outputs (1,1) and (0,0) are summed up to get the new output (1,1). Then, shift-and-add of the new outputs generated by the first step is calculated based on the bit-position of the input feature map to get the new result (3,5) in the second step. At the last step, shift-and-add is performed based on the bit-position of the filter, in order to get the final error-included dot-product result ($3 \times 2^1 + 5 \times 2^0 = 11$). In this example, the original output, 9 (Figure 6(a)), is changed to 11, due to the errors induced by unstable ReRAM cells, accumulated currents at bitlines within an OU, and non-ideal SA. To analyze the impact of these hardware-induced errors on the inference accuracy of neural networks,

a simulation platform which integrates the modeling of error rates per bitline with the OU-based dot-product computation process is needed.

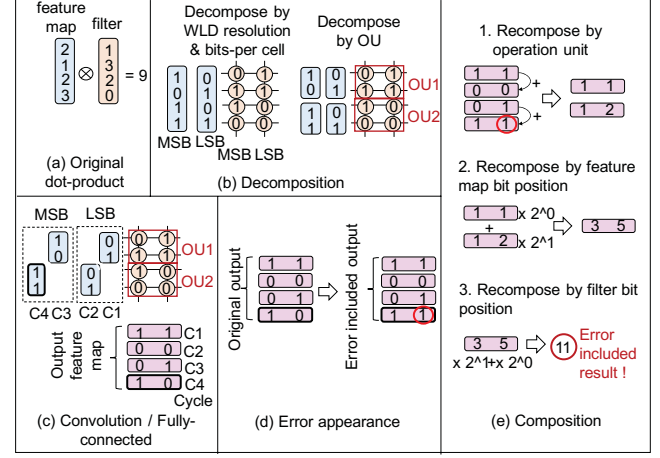


Figure 6: An example of OU-based dot-product computation.

4 DL-RSIM: SIMULATION FRAMEWORK

DL-RSIM is a simulation framework that enables the analysis on the impact of non-ideal circuit and device properties on the inference accuracy for memristor-based deep learning accelerators. Figure 7 shows the simulation framework of our DL-RSIM, which is composed of two modules. The **NVM Error Analytical Module** takes a set of hardware configurations (e.g., ReRAM cell properties, SA-related parameters, etc) as input parameters, and estimates the error rates of each sum-of-products computation per bitline within an OU. The **TensorFlow-based Inference Accuracy Simulation Module** is based on a typical TensorFlow inference flow and incorporates the underlying ReRAM-based hardware configuration and estimated error rates of an OU from the **NVM Error Analytical Module** to generate inference accuracy for the target input feature map and neural network model. All the input hardware configurations are listed in Table 1 and the detailed designs of each module are described in the following subsections.

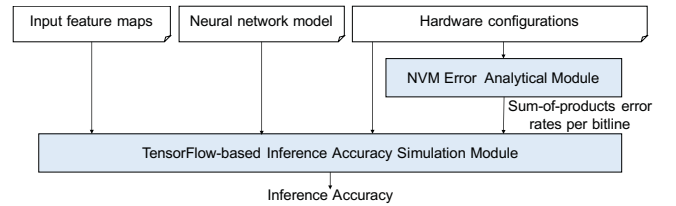


Figure 7: Simulation framework of DL-RSIM.

4.1 NVM Error Analytical Module

The NVM Error Analytical Module estimates the error rates of each sum-of-products results per bitline within an OU, considering the non-ideal ReRAM cell and SA. Figure 8 shows the simulation flow of this module. First, given the number of bits per cell, R_{on} , R-ratio, and resistance deviation (σ), we can build the cell resistance distribution, which is modeled by the **Probability Density Function (PDF)** of lognormal

Table 1: Input hardware configurations of DL-RSIM.

Name	Description
WLD resolution	Number of bits to encode input voltage levels
Bits-per-cell	Number of bits stored in each cell
Cell R_{on}	Cell resistance of on-state
Cell R-ratio	Cell resistance ratio between the highest state and the lowest state
Cell resistance deviation (σ)	Cell resistance deviation of each state
SA resolution	Number of output bits per SA
SA $I_{R[n]}$	SA reference current of each state e.g., mean of each state
SA dV_{SA}	SA safety guard band delta voltage
OU size (S_{OU})	Max number of wordlines that can be activated concurrently within a crossbar array

distribution [6], from low-resistance to high-resistance states:

$$P(R_i) = \frac{1}{R_i \sigma_i \sqrt{2\pi}} * \exp\left(-\frac{(\log(R_i) - \mu_i)^2}{2(\sigma_i)^2}\right) \quad (4)$$

where μ_i is the mean resistance and σ_i is the resistance deviation of state i . $\mu_i = R_{on}$ at the lowest-resistance state and $\mu_i = R_{on} * R\text{-ratio}$ at the highest-resistance state. According to the Ohm's law, we can use Jacobian to get the **PDF** of cell current distribution, which is modeled by:

$$P(I_i) = \frac{1}{I_i \sigma_i \sqrt{2\pi}} * \exp\left(-\frac{(\log(\frac{V_r}{I_i}) - \mu_i)^2}{2(\sigma_i)^2}\right) \quad (5)$$

where V_r is the bitline clamping voltage¹.

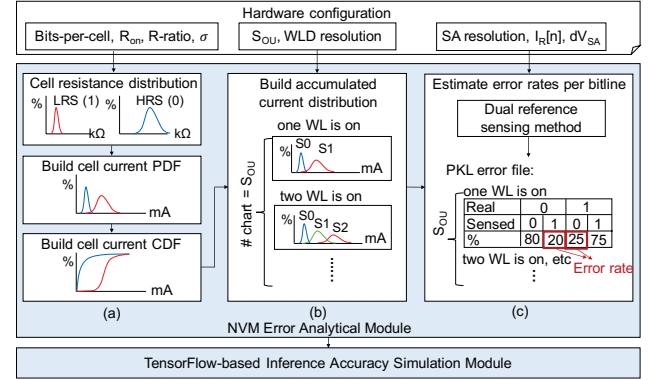
We then transform **PDF** into **Cumulative Distribution Function (CDF)**, so that we can use the Monte Carlo sampling method to get the accumulated current distribution (i.e., sum-of-products results) within an OU, as shown in Figure 8(b). In an OU, it is possible to activate 1 to S_{OU} wordlines, depending on the number of non-zero inputs. Since the accumulated current distribution varies when different number of wordlines within an OU are activated, we generate one chart for each possible number of activated wordlines. Each chart of accumulated current distribution is derived by using the Monte Carlo sampling method. For example, if N wordlines are activated within an OU when WLD resolution = 1 and bits-per-cell = 1, there are $N+1$ possible sum-of-products results (0 to N). For each possible sum-of-products result, we use Monte Carlo to get current samples from the CDF of HRS and LRS, and add these currents together. For instance, if sum-of-products = 2, we sum up the current of two samples from the CDF of LRS and $N-2$ samples from the CDF of HRS. This sampling process is repeated several times to accurately estimate the accumulated current distribution².

Finally, we estimate the error rates of each sum-of-products results based on the selected SA sensing scheme (e.g., SA resolution, SA reference current, dV_{SA} , etc), as shown in Figure 8(c). When dual reference sensing scheme is applied, Equation 2 is used to determine whether each sample in the accumulated current distribution is correctly read out. We generate an error table for each possible number of activated

¹In this paper, $V_r = 0.6V$ + bitline clamping error, and the bitline clamping error is less than 0.03V.

²We sample 32000 times for each possible sum-of-products result in this paper. The resistance of the bitline itself is also included when calculating the accumulated current.

wordlines within an OU, as the number of activated wordline is input-dependent and is unknown at this stage. For each possible sum-of-products result k , we calculate the percentage of sensing k as all possible values, from 0 to the total number of activated wordlines. For example, when one wordline is activated and real sum-of-products result is 0, the percentage of being sensed as 0 and 1 are both recorded in the table. The estimated error rates are written into a Python Pickle File (PKL file), which is easy to dump and load into programs. The PKL file is passed to the TensorFlow-based Inference Accuracy Simulation Module, in order to analyze the impact of current sensing errors on the inference accuracy.


Figure 8: The flow of NVM Error Analytical Module.

4.2 TensorFlow-based Inference Accuracy Simulation Module

TensorFlow-based Inference Accuracy Simulation Module models the impact of accumulated current sensing errors per bitline within an OU on the inference accuracy of the targeted neural network. Figure 9 shows the simulation flow of this module. Figure 9(a) is a typical TensorFlow inference flow with two convolution layers and one fully-connected layer. We modify convolution and fully-connected layers to incorporate the errors induced by non-ideal circuit and device properties into the neural network, as shown in Figure 9(b). Three additional functions, **Decomposition**, **Error Injection**, and **Composition**, are added into the convolution and fully-connected layers, in order to perform OU-based computation and integrate with the current sensing error rates per bitline within an OU provided by the NVM Error Analytical Module. The error-injected results are then compared with the ideal results to get the inference accuracy, as shown in Figure 9(c). In the following, we use the OU-based dot-product computation example in Figure 6 to describe the modified convolution and fully-connected layers in detail.

Decomposition. The goal of decomposition is to partition original feature maps and filters, so that we can compute the dot-product results in an OU-based manner. Figure 10 shows the flow of Decomposition. First, original feature maps from the input or pooling layer are transformed to fixed-point representation using the same method introduced by Shafiee et al. [9], and these feature maps are decomposed based on the WLD resolution. The MSB to LSB bits of each data in the feature maps are separated into different groups. The original

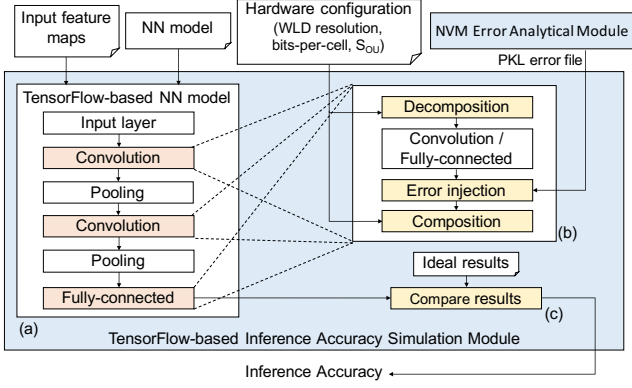


Figure 9: The flow of TensorFlow-based Inference Accuracy Simulation Module.

filters are also transformed to fixed-point representation and are separated into MSB to LSB groups by the underlying bits-per-cell. Considering that limited number of wordlines can be activated concurrently within a crossbar array, we further partition the feature maps and filters based on which OU is responsible for the sum-of-products computation. The example in Figure 6(b) shows that, by applying the Decomposition process, the original input feature map is separated into four groups based on bit position and S_{OU} , while the original filter spans across two bitlines and is partitioned into two OU-based groups (OU1 and OU2). After the Decomposition process, we can use the original Convolution/Fully-connected function in TensorFlow to perform OU-based dot-product computation and pass the output feature maps to the newly added Error Injection function to generate error-included output data, considering the accumulated current sensing errors per bitline within an OU.

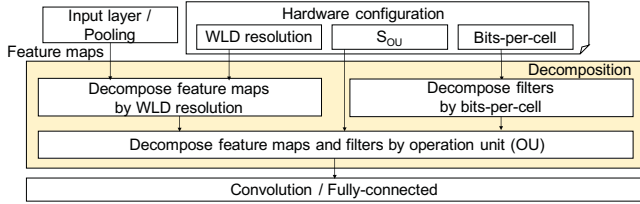


Figure 10: The flow of Decomposition.

Error Injection. To analyze the impact of accumulated current sensing errors per bitline within an OU on the inference accuracy, we need to inject the error rates provided by the NVM Error Analytical Module. First, for each data in the original output feature maps, we index the PKL error file to get the probability of each possible sensing value, as shown in Figure 11(a). For example, assume that the second bit in the fourth output data is 0. By indexing the PKL error file, we know that the probability of sensing 0, 1, 2, are 90%, 8%, and 2%, respectively. Figure 11(b) shows how to inject errors in the outputs based on the indexed error rate. We first use a random number generator to generate a number, A , that falls into the range between 0 and 100. We then compare A with the probability of each possible sensing value to get the error-injected data. In this example, A is smaller than 10 but larger than 2, so the original output 0 is changed to 1 based

on the error rate. The error-injected output feature maps are then passed to the Composition function.

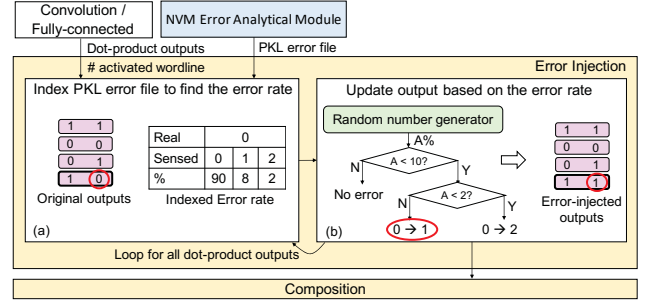


Figure 11: The flow of Error Injection.

Composition. After the errors are injected, we recompose these OU-based bit-level data into the final dot-product outputs. Figure 12 shows the flow of Composition. First, based on OU size, the outputs of the OUs that share the same set of bitlines are added together. We then perform shift-and-add based on the bit-position of the feature map, followed by the shift-and-add based on the bit-position of the filter. The example in Figure 6(e) shows that, by applying the Composition process, we can resemble the OU-based bit-level outputs (1,1), (0, 0), (0,1), (1,1), into the final error-included result (11). The error-injected and recomposed output feature maps are then transformed from fixed-point representation to floating-point representation and passed to the next TensorFlow pooling layer.

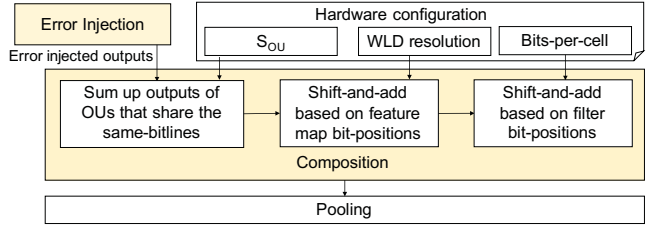


Figure 12: The flow of Composition.

5 EXPERIMENTAL RESULTS

In this section, we first validate the correctness of our DL-RSIM by comparing the modeled current distribution with SPICE. We then use three representative CNNs as case studies to show that DL-RSIM can help chip designers to select reliability-friendly design options and develop reliability optimization techniques to reduce error rates.

5.1 Validation

We use the simulation results from SPICE to validate the correctness of DL-RSIM. With the same LRS and HRS resistance distribution [6], we compare the accumulated current distribution derived from SPICE and DL-RSIM when $S_{OU} = 9$. When simulating the accumulated current on a bitline, SPICE models the interference between cells, which is not modeled in DL-RSIM. Figure 13 shows that the accumulated current distribution generated by our analytical module is similar to SPICE. The root mean square error between these two accumulated current distributions is less than 0.5%.

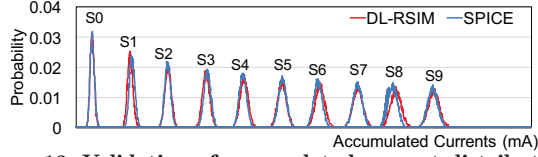


Figure 13: Validation of accumulated current distribution.

5.2 Case studies

Three representative picture classification and face recognition datasets, including **MNIST**, **CIFAR-10** and **Youtube Face (YTF)**, are utilized to show the impact of different hardware configurations on the inference accuracy by using our DL-RSIM. These experiments demonstrate how DL-RSIM can guide chip designers to choose better design options at an early design stage. Table 2 lists the neural network model for these datasets. Based on a practical ReRAM-based accelerator chip [2], we choose $S_{OU} = 9$, WLD resolution=1, and single-level cell as our baseline configuration in the following evaluation. A dual reference sensing scheme with SA resolution = 4 bits and $dV_{SA} = 0$ is used in these experiments except when exploring the impact of dV_{SA} in Section 5.2.4.

Table 2: Neural network model for the target datasets. (IL: integer length; FL: fractional length)

Name	convolution	fully connected	data precision	original accuracy
MNIST	1 layer	2 layers	<IL=2,FL=4>	99.99%
CIFAR-10	3 layers	3 layers	<IL=3,FL=6>	88.67%
YTF	4 layers	2 layers	<IL=3,FL=6>	90.53%

5.2.1 R-ratio. We first show the impact of R-ratio on the inference accuracy by varying the R-ratio from 10 to 50, with fixed R_{on} and resistance deviation (σ) setting³. Figure 14(a) shows that the error rate of each sum-of-products result decreases as the R-ratio increases, since increasing R-ratio reduces the overlapped region between LRS and HRS resistance distribution. For example, the error rate of 0 decreases from 25% to 0% when the R-ratio increases from 10 to 50. The decrease in error rates per bitline leads to the improvement of inference accuracy, as shown in Figure 14(b). For MNIST, the error-injected inference accuracy reaches the original accuracy (99.99%) when R-ratio ≥ 25 . However, CIFAR-10 and YTF can achieve only 59.33% and 62.00% inference accuracy even when R-ratio = 50, as these two models are deeper than the model of MNIST and the error propagation at deeper networks exacerbates the accuracy degradation problem. These results imply that chip designers can *choose an ReRAM cell with higher R-ratio to improve inference accuracy and deeper neural networks require larger R-ratio to avoid serious accuracy loss*.

5.2.2 Variance of cell resistance. In addition to cell R-ratio, we also evaluate the impact of various cell resistance deviation (σ) on the inference accuracy. We choose the σ in prior work [6] as the baseline setting (σ_b) and change σ from σ_b to $1/5 \sigma_b$. ReRAM cell designers can use device-level techniques, such as program-and-verify, to reduce σ , and

³ R_{on} and σ are set to the same as [6].

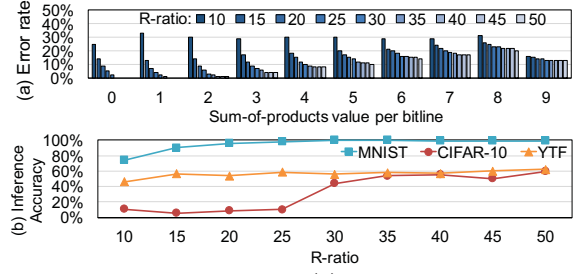


Figure 14: Impact of R-ratio on (a) error rates per bitline and (b) inference accuracy.

smaller σ implies smaller tail regions in the cell resistance and current distribution. As shown in Figure 15(a), the sensing error rate per bitline decreases as σ decreases, since the overlapped region between neighbor states in the accumulated current distribution becomes smaller. Figure 15(b) shows that smaller deviation leads to higher inference accuracy. When R-ratio=10, the inference accuracy of MNIST is similar to the hardware-error-free inference accuracy (99.99%) when $\sigma \leq 3/5 \sigma_b$, while the deeper CIFAR-10 and YTF needs smaller σ ($2/5 \sigma_b$) to eliminate accuracy loss. When R-ratio is larger (R-ratio=50), small reduction in σ is enough to achieve high inference accuracy. For example, CIFAR-10 can achieve 86.00% accuracy by reducing the σ to $2/5 \sigma_b$ when R-ratio=10, while reducing σ to $3/5 \sigma_b$ is enough to achieve 87.33% accuracy when R-ratio=50. Note that reducing σ can achieve an accuracy similar to the hardware-error-free accuracy for all datasets. Comparing Figure 15 and Figure 14, we can find that *reducing cell resistance deviation can help to improve the inference accuracy, and it is more effective than increasing the R-ratio*.

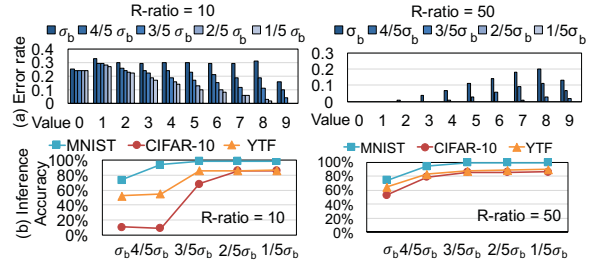


Figure 15: Impact of resistance deviation on (a) bitline error rate and (b) inference accuracy.

5.2.3 Operation unit. We also evaluate the impact of various S_{OU} on the inference accuracy, as shown in Figure 16. In this experiment, $\sigma = \sigma_b$ but different R-ratios are chosen for different datasets, so that we can start with a baseline ($S_{OU} = 9$) setting that has room for accuracy improvement and degradation (i.e., the accuracy that is imperfect but not too low). Smaller S_{OU} can help to alleviate the aggregation of cell resistance variance when performing dot-product computations with many non-zero inputs. Larger S_{OU} can provide higher degree of parallelism and better performance, but would significantly degrade the inference accuracy. Figure 16 shows that the inference accuracy increases when smaller S_{OU} is applied, especially at the datasets with deeper neural network (CIFAR-10 and YTF). These results imply that if

existing cell R-ratio and σ fail to provide an acceptable inference accuracy for the targeted deep learning applications, *reducing the number of concurrently activated wordlines can also help to improve the inference accuracy.*

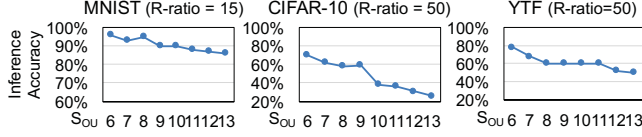


Figure 16: Impact of operation unit size on inference accuracy.

5.2.4 Safety guard band delta voltage. Figure 17 shows the impact of dV_{SA} on inference accuracy. We fix cell resistance (R -ratio=40 and $\sigma=4/5 \sigma_b$) and vary dV_{SA} from 0 to 50mV in this experiment. Results show that the accuracy decreases as dV_{SA} increases, since a SA with larger dV_{SA} is more likely to sense a wrong value. For MNIST, $dV_{SA} \leq 35mV$ is enough to provide an accuracy similar to the hardware-error-free accuracy. However, for deeper and more complex CIFAR-10 and YTF, dV_{SA} must be lower than 20mV and 5mV to provide acceptable accuracy. This implies that *choosing a SA with smaller dV_{SA} can help to improve the inference accuracy and deeper neural networks require smaller dV_{SA} .*

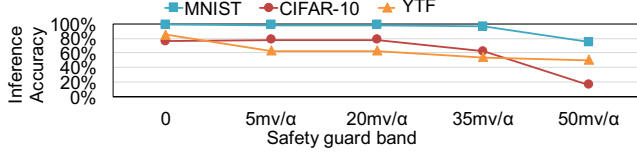


Figure 17: The impact of dV_{SA} on inference accuracy.

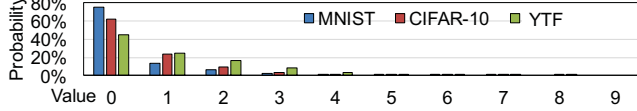


Figure 18: The appearance rates of different sum-of-products values when $S_{OU}=9$.

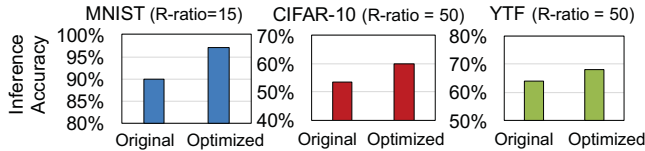


Figure 19: Inference accuracy improvement provided by workload-dependent optimization on the sensing scheme.

5.2.5 Workload-dependent sensing scheme. We propose and evaluate a workload-dependent sensing scheme to show that chip designers can use DL-RSIM to develop reliability optimization techniques for ReRAM-based deep learning accelerators. Figure 18 shows that small values (<5) occur more often than large values (≥ 5) when performing OU-based dot-product computation in all these three datasets. In particular, more than 40% of sum-of-products results per bitline are zero due to the sparsity of feature maps and synaptic weights in these datasets, especially MNIST. Thus, we change the offset of the reference current when distinguishing 0 from 1 ($I_R[0]$) to increase the opportunities of reading out 0. Figure 19 shows that through shifting $I_R[0]$ to right by 0.000397 and 0.000265 mA for MNIST and

CIFAR-10, we can greatly improve the inference accuracy. For YTF, we shift $I_R[0]$ (resp. $I_R[1]$) to right by 0.000265 (resp. 0.000397) mA, as many sum-of-products result per bitline are zero and one.

6 RELATED WORK

The two most well-known simulation platforms for ReRAM-based neural network accelerators are MNSIM [12] and NeuroSim [1]. MNSIM is a behavior-level simulation platform that provides flexible interface for users through a hierarchical module structure, and it can be utilized to estimate the area, power, latency, and inference accuracy. Nevertheless, they use only worst and average case for simulation, resulting in less precise estimation on inference accuracy. NeuroSim is a circuit-level simulation tool that takes the impact of non-ideal device and circuit properties into consideration when estimating the area, power, and performance of ReRAM-based accelerators. However, NeuroSim's architecture is less flexible, since it is fixed on a 2-layer multi-level perceptron (MLP) neural network. Comparing to these prior studies, our DL-RSIM can provide detailed analysis on inference accuracy under different hardware configurations and support flexible simulation for any TensorFlow-based neural network models.

7 CONCLUSION

We propose DL-RSIM, a flexible simulation framework, to precisely explore the impact of non-ideal circuit and device properties on the inference accuracy of various neural networks. DL-RSIM can help chip designers to choose reliability-friendly options and develop reliability optimization techniques based on DL-RSIM's analysis. In the future, we plan to add performance and power estimation modules in DL-RSIM to enable more comprehensive design space exploration.

REFERENCES

- [1] P. Y. Chen et al. 2018. NeuroSim: A Circuit-Level Macro Model for Benchmarking Neuro-Inspired Architectures in Online Learning. *IEEE TCAD* (2018), 1–1.
- [2] W. H. Chen et al. 2018. A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors. In *ISSCC*. 494–496.
- [3] P. Chi et al. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *ISCA*. 27–39.
- [4] H. Esmailzadeh et al. 2012. Neural Acceleration for General-Purpose Approximate Programs. In *MICRO*. 449–460.
- [5] Ben Feinberg et al. 2018. Making Memristive Neural Network Accelerators Reliable. In *HPCA*. 52–65.
- [6] K.C. Hsu et al. 2015. A Study of Array Resistance Distribution and a Novel Operation Algorithm for WOx ReRAM Memory. In *SSDM*.
- [7] M. Hu et al. 2016. Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication. In *DAC*. 1–6.
- [8] S. W. Keckler et al. 2011. GPUs and the Future of Parallel Computing. *IEEE Micro* (2011), 7–17.
- [9] A. Shafiee et al. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *ISCA*. 14–26.
- [10] F. Su et al. 2017. A 462GOPS/J RRAM-based nonvolatile intelligent processor for energy harvesting IoE system featuring nonvolatile logics and processing-in-memory. In *VLSI-TSA*. T260–T261.
- [11] H. S. P. Wong et al. 2012. Metal-Oxide RRAM. *Proc. IEEE* (2012), 1951–1970.
- [12] L. Xia et al. 2017. MNSIM: Simulation Platform for Memristor-based Neuromorphic Computing System. *IEEE TCAD* (2017), 1–1.